RAPPORT TECHNIQUE FIN DE PROJET

Projet transversal n°4

Collecteur de données animalières

Équipe projet :
Davy Clark, Ewen Le Callet, Joris Menard, Jules Noumba, Xueying Xu

Client :
Xavier Leprevost – Élu de la mairie de Saint-Mars-du-Désert

Encadrants :
Abdelhakim Saadane et Antoine Goullet

Date :
09/03/2026

Année universitaire 2025–2026

Remerciements

Nous tenons à remercier l'ensemble des personnes ayant contribué à la réalisation de ce projet transversal. Nous remercions particulièrement nos encadrants, Abdelhakim Saadane et Antoine Goullet, pour leur accompagnement, leurs conseils méthodologiques et leur disponibilité tout au long de l'année universitaire 2025–2026.
Nous remercions également notre client, Xavier Leprevost, élu de la mairie de Saint-Mars-du-Désert, pour sa confiance, la clarté de ses besoins et la qualité des échanges durant le développement du *collecteur de données animalières*.

Table des matières

Remerciements **1**

Table des figures **4**

Glossaire **5**

1 Introduction **6**

2 Présentation, contexte et spécifications **6**
2.1 Présentation de l'entreprise 6
2.2 Contexte général 6
2.3 Présentation du sujet de projet et Cahier des Charges 6
2.4 Etat de l'art 6

3 Liste des Work-Packages : Objectifs, Solution, Validation **6**
3.1 WP 1.1 — Détection de mouvement et mesure de température (responsable : Jules Noumba) 6
3.1.1 Rappel des objectifs 6
3.1.2 Solution envisagé 6
3.1.3 Tests et Validation 6
3.2 WP 1.2 — Gestion des échanges données et du microcontrôleur (responsable : Nolan Buchet) 7
3.2.1 Rappel des objectifs 7
3.2.2 Solution envisagé 7
3.2.3 Tests et Validation 11
3.3 WP 1.3 — Caméra (responsable : Xueying Xu) 12
3.3.1 Rappel des objectifs 12
3.3.2 Solution envisagée 12
3.3.3 Tests et Validation 14
3.4 WP 1.4 — Conception et fabrication du boitier (responsable : Joris Menard) 15
3.4.1 Rappel des objectifs 15
3.4.2 Solution envisagé 15
3.4.3 Tests et Validation 15
3.5 WP bis — Transmission (responsable : Joris Menard) 15
3.5.1 Rappel des objectifs 15
3.5.2 Solution envisagé 15
3.5.3 Tests et Validation 15
3.6 WP 2.1 — Stockage distant (responsable : Davy Clark) 15
3.6.1 Rappel des objectifs 15
3.6.2 Solution envisagé 15
3.6.3 Tests et Validation 15
3.7 WP 2.2 — Interface web (responsable : Xueying Xu) 15
3.7.1 Rappel des objectifs 15
3.7.2 Solution envisagé 16
3.7.3 Tests et Validation 16
3.8 WP 2.3 — Classification animalière par IA (responsable : Ewen Le Callet) 16
3.8.1 Rappel des objectifs 16

3.8.2 Solution envisagé 16
3.8.3 Tests et Validation 16

4 Bilan du projet **16**
4.1 Bilan technique 16
4.2 Bilan personnel 16

5 Conclusion **16**

Bibliographie **17**

Annexes **18**

A Répartition détaillée des Work-Packages **18**

B Exemples de données collectées **18**

Table des figures

1 Pinout Diagramme de l'ESP32 9
2 Module de carte SD utilisé pour le projet 10
3 Module de carte SD utilisé pour le projet 11
4 Chaîne d'acquisition et de traitement des images du module caméra 13
5 Influence de la résolution sur la taille des images JPEG et le temps de capture 14

Glossaire

WP Un Work Package (WP) est une unité de travail dans un projet. Il regroupe un ensemble cohérent de tâches, d'objectifs et de livrables.

1 Introduction

Le projet **Collecteur de données animalières** s'inscrit dans le cadre du projet transversal 2025–2026 mené par des étudiants de Polytech Nantes en collaboration avec la mairie de Saint-Mars-du-Désert.
L'objectif principal est de concevoir une solution permettant de collecter, structurer et exploiter des données relatives à la faune locale, afin d'améliorer le suivi du territoire et d'appuyer la prise de décision.
Ce rapport technique présente le contexte du projet, les besoins exprimés par le client, les choix techniques retenus, ainsi que l'avancement des travaux réalisés par l'équipe. Par ailleurs, la structuration en lots de travail s'inspire des bonnes pratiques de gestion de projet recommandées dans la littérature[1]. Par ailleurs, la structuration en lots de travail s'inspire des bonnes pratiques de gestion de projet recommandées dans la littérature[1].

2 Présentation, contexte et spécifications

2.1 Présentation de l'entreprise

2.2 Contexte général

2.3 Présentation du sujet de projet et Cahier des Charges

2.4 Etat de l'art

3 Liste des Work-Packages : Objectifs, Solution, Validation

Parler de la décomposition des WP, sous WP etc

3.1 WP 1.1 — Détection de mouvement et mesure de température (responsable : Jules Noumba)

3.1.1 Rappel des objectifs

Détailler objectifs et liste des taches.

3.1.2 Solution envisagé

Travail théorique et recherches

3.1.3 Tests et Validation

Tests et validation de vos taches.

3.2 WP 1.2 — Gestion des échanges données et du microcontrôleur (responsable : Nolan Buchet)

3.2.1 Rappel des objectifs

Pour recontextuatlisé le projet et les objectifs de ce Work Package, le WP 1.2 fait partie du Work parckage 1 qui est dédié à la conception et au développement du système de collecte de données animalières. Le WP 1.2 se concentre spécifiquement sur la gestion des échanges de données entre les différents composants du système, ainsi que sur l'intégration et la programmation du microcontrôleur qui servira de cerveau central pour le projet. De plus celui-ci s'occupera de la partie énergétique du module, ainsi que de la partie stockage des données collectées.

Voici la liste des taches qui seront réalisées dans le cadre de ce WP :
- Tache 0 (Equipe) : Choix du Microcontrôleur et de la carte SD
- Tache 1 : Gestion du stockage des données collectées
- Tache 2 (hors WP) : Gestion de la luminosité pour WP 1.3
- Tache 3 ; Gestion de l'horodatage et de la localisation du modules
- Tache 4 : Définition d'un Automate pour la gestion global du module
- Tache 5 : Développement d'une architecture electrique
- Tache 6 : Intégration des modules (prototypage)
- Tache 7 : Gestion de l'alimentation du module
- Tache 8 : Création d'un PCB pour le module

Pour chacune des taches réalisée ci-dessus, il y a eu une phase de test réalisé, ainsi qu'en amont une phase de concertation avec l'équipe du WP afin que les solutions envisagées soient en adéquation avec les besoins du projet.

Certaines tâches de ce modules ne sont pas dans l'ordre de conception, en prenant la tache 4 ou 5 qui ont été en constante évolution durant tout le projet. Et certaine taches non essentielle, mais logique, comme l'optimisation des codes, ou rapport / documention ont été réalisé en parallèle de la conception du module.

3.2.2 Solution envisagé

Tache 0 : Choix du Microcontrôleur

À la suite d'une phase de recherche (benchmark du début de projet) concernant le contrôleur à utiliser pour ce projet, nous avons identifié deux solutions susceptibles de répondre à nos besoins : l'ESP32 et le Raspberry Pi Zero.

Pour rappel, les besoins principaux concernant le contrôleur étaient les suivants :

Critère	Description
Consommation énergétique	Minimiser la consommation pour un usage sur batterie.

Performances	Garantir une puissance suffisante pour le traitement des données.
Stockage	Disposer d'une capacité suffisante pour enregistrer temporairement les mesures locales et la photographie.
Système	Offrir un environnement logiciel simple avec facilité sur les outils de développement utilisés.
Interfaces	Proposer des entrées/sorties adaptées aux capteurs et modules externes.
Connectivités	Intégrer ou supporter le Wi-Fi, le Bluetooth ou d'autres moyens de communication.
Prix	Rester dans une gamme de coût compatible avec le budget du projet.

Ces critères nous ont permis de comparer plusieurs solutions techniques lors de la phase de benchmark.

Les critères suivants ont été considérés lors du choix final du microcontrôleur :
- Architecture matérielle
- Connectivité
- Consommation énergétique
- Nombre et type de GPIO
- Facilité de développement
- Coût global

À l'issue de l'analyse comparative réalisée entre l'ESP32 et le Raspberry Pi Zero, le choix s'est porté sur l'ESP32 comme microcontrôleur principal pour le collecteur de données animalières. Ce choix est motivé par plusieurs facteurs clés. Tout d'abord, sa faible consommation énergétique constitue un avantage décisif pour une utilisation sur batterie dans un environnement isolé. L'ESP32 peut fonctionner plusieurs jours, voire semaines, en mode basse consommation, ce qui garantit une autonomie optimale sur le terrain.

Ensuite, son intégration native du Wi-Fi et du Bluetooth simplifie la conception matérielle et réduit le besoin de modules externes, tout en permettant la transmission sans fil des données et la configuration à distance du système.

Enfin, son coût réduit et sa disponibilité sur le marché en font une solution économique et accessible, sans compromis majeur sur les performances nécessaires au projet. En revanche, il est à noter que le Raspberry Pi Zero offre une puissance de calcul et une mémoire supérieures, mais celles-ci sont jugées excessives au regard des besoins du projet et pénalisantes en termes de consommation énergétique et de gestion logicielle.

En conclusion, l'ESP32 répond pleinement aux exigences du projet en matière de consomma tion, de connectivité et de simplicité de déploiement, ce qui en fait la solution la plus adaptée pour le

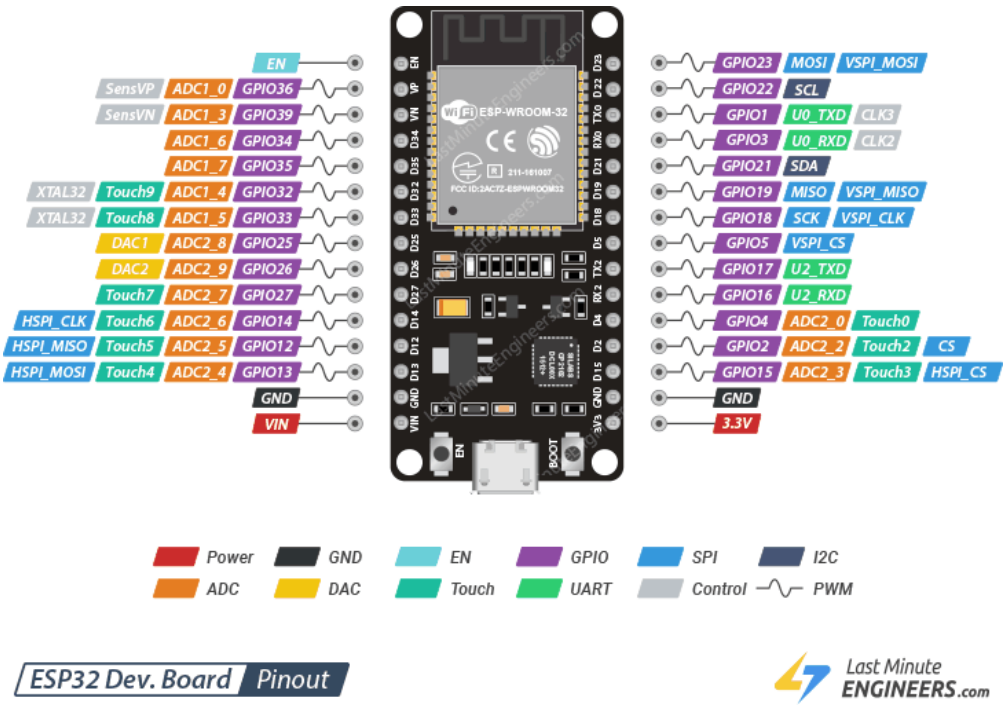

FIGURE 1 – Pinout Diagramme de l'ESP32

collecteur de données animalières.

Notes : Un document détaillé présentant l'analyse comparative entre l'ESP32 et le Raspberry Pi Zero, ainsi que les critères de sélection, à été rédigé.

Tache 1 : Gestion du stockage des données collectées

Commençons par la tache 1 qui est la gestion du stockage des données collectées. Pour cette tache, il a été décidé d'utiliser une carte SD (par cahier des charges) pour stocker les données collectées par les métadatas et images. La carte SD offre une grande capacité de stockage, ce qui est essentiel pour notre projet De plus, les cartes SD sont relativement faciles à interfacer avec un microcontrôleur, ce qui facilite l'intégration dans notre système. Enfin celle ci est facilement accessible pour les utilisateurs, ce qui permet une récupération facile des données collectées.

Dons nous avons utilisé un module de carte SD utilisable en moyen de communication SPI, qui est un protocole de communication série rapide et efficace. Le microcontrôleur peut ainsi envoyer des commandes à la carte SD pour lire et écrire des données de manière rapide et fiable. Le module utilisé est un module personnel (`AZDelivery SPI Reader`), celui-ci à été choisi au début du projet car aucune commande n'étais envisageable de la part du département ou du client.

Phase d'implémentation

Pour utilisé le module, nous avons connecté notre esp32 à ce module en liaison SPI, en utilisant les broches suivantes :

— MOSI (Master Out Slave In : Écriture) : GPIO 13

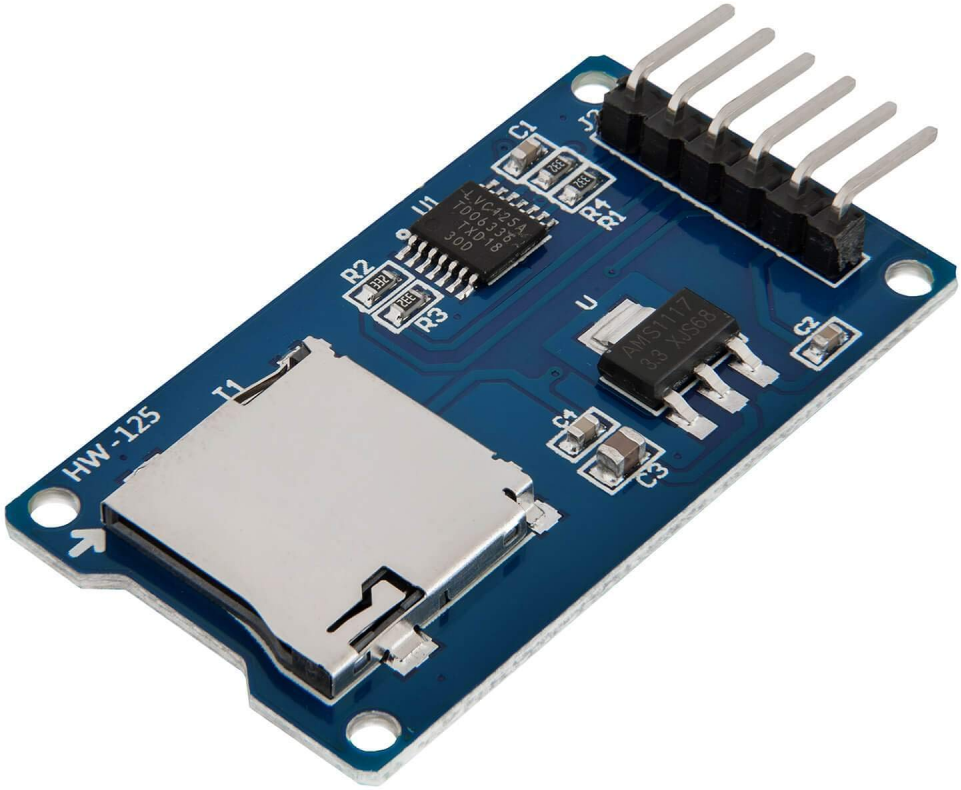

Figure 2 – Module de carte SD utilisé pour le projet

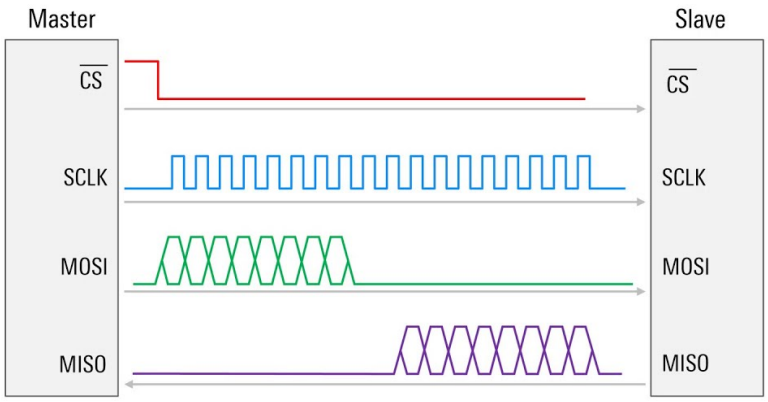

FIGURE 3 – Module de carte SD utilisé pour le projet

- MISO (Master In Slave Out : Lecture) : GPIO 12
- SCK (Serial Clock) : GPIO 14
- CS (Chip Select) : GPIO 15
- VCC : 5V
- GND : GND

Nous utiliserons le module HSPI de l'ESP32 pour la communication SPI, qui permet des vitesses de transfert élevées. Voici un exmple de communication en SPI pour décrire le moyen de communication entre l'ESP32 et la carte SD.
Pour utiliser ce module, une librairie toute faite à déja été développé, qu'il à suffit d'implémenter dans notre projet pour pouvoir utilisé la carte SD. Cette librairie est la librairie `SD.h` qui est une librairie standard pour la gestion des cartes SD avec les microcontrôleurs Arduino et ESP32. Elle offre des fonctions pour initialiser la carte SD, lire et écrire des fichiers, et gérer les répertoires. Nous l'avons utilisé pour crée ntre propre bibliothèque visant à simplificer l'utilisation de cette carte avec notement le WPBis qui est dédier à la gestion de la transmission des données collectées.

Phase de Test

Pour la phase de test, nous avons juste réalisé un test afin de savoir si nous pouvions écrire un fichier texte sur la carte SD, En donnant comme Nom de fichier les métadatas (comme température, humidité, etc), afin de simulé un buffer de réception d'images avec les données correspondantes.

Voici un exemple de code utilisé pour ce test :

3.2.3 Tests et Validation

3.3 WP 1.3 — Caméra (responsable : Xueying Xu)

3.3.1 Rappel des objectifs

Le work package WP1.3 correspond au module caméra du système.
Son objectif principal est de permettre la prise d'images dans le cadre du piège caméra, afin de récupérer des données exploitables sur les animaux détectés.
Plus concrètement, les tâches associées sont :

- Capturer une image lorsqu'un signal de déclenchement est reçu (capteur ou commande)
- Générer des images dans un format adapté, notamment le format JPEG
- Assurer un fonctionnement en conditions de jour comme de nuit, avec un mode nocturne basé sur un éclairage infrarouge
- Obtenir une qualité d'image suffisante pour permettre une observation correcte
- Assurer que les images puissent être utilisées par les autres modules, en particulier le module MCU (WP1.2) et l'interface web (WP2.2)
- Étudier l'influence de certains paramètres (résolution, distance, etc.) sur la qualité et la taille des images

Au début du projet, plusieurs solutions ont été testées (Raspberry Pi, ESP32-CAM) afin de valider le fonctionnement global. La solution matérielle finale étant traitée dans le WP1.2, cette partie se concentre surtout sur la gestion des images et leur format.
Dans ce contexte, nous avons mené une étude spécifique sur le choix du format d'image ainsi que sur son impact sur le système global. En particulier, le format JPEG a été analysé en lien avec la taille des données, la qualité de l'image et les contraintes de transmission. Parallèlement, nous avons également réalisé plusieurs tests afin d'évaluer l'influence de la résolution et des conditions de prise de vue sur le module caméra et la qualité de l'image.

3.3.2 Solution envisagée

Afin de répondre aux objectifs du WP1.3, une solution basée sur l'acquisition et la gestion d'images au format JPEG a été retenue.
Une phase de prototypage a d'abord été réalisée à l'aide de différentes plateformes (Raspberry Pi puis ESP32-CAM) afin de valider le principe de capture d'image et de transmission. La solution matérielle finale étant traitée dans le WP1.2, cette partie se concentre principalement sur le traitement des images.
La chaîne de fonctionnement mise en place est illustrée sur la figure ci-dessous. Elle comprend le déclenchement de la capture, l'acquisition de l'image, la génération d'une image compressée au format JPEG, puis son stockage temporaire avant exploitation par les autres modules du système.

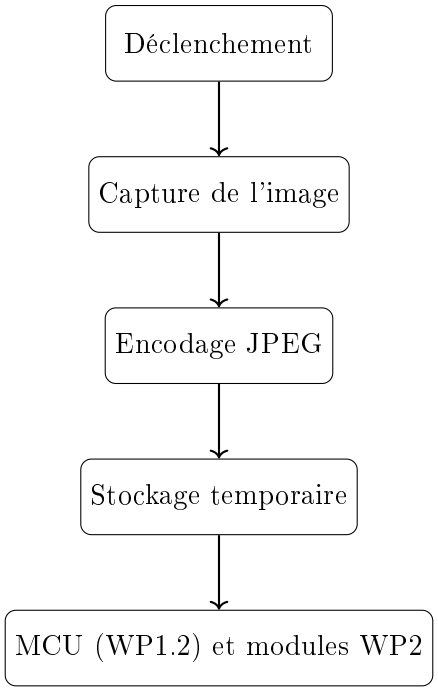

FIGURE 4 – Chaîne d'acquisition et de traitement des images du module caméra

Choix du capteur OV2640 Le capteur OV2640 a été retenu car il est bien adapté à une application embarquée de faible coût. Il est compatible avec l'ESP32 et permet de produire directement des images exploitables dans le système.
Son fonctionnement repose sur la conversion de la lumière captée par le capteur en données numériques. Ces données peuvent ensuite être traitées afin de générer une image compressée au format JPEG, qui sera récupérée par le microcontrôleur pour être stockée ou transmise.
Ce choix présente plusieurs avantages dans le cadre du projet :

- génération directe d'images JPEG ;
- réduction de la charge de traitement côté microcontrôleur ;
- taille de données plus adaptée à la transmission ;
- bonne intégration dans une architecture embarquée.

Principe du format JPEG Le format JPEG repose sur une compression avec perte, ce qui permet de réduire fortement la taille des images. Son principe est de représenter l'image en séparant les informations les plus importantes de celles qui le sont moins pour l'œil humain.
D'un point de vue théorique, l'image est découpée en blocs, puis chaque bloc est transformé dans le domaine fréquentiel, généralement par une transformée en cosinus discrète (DCT). Les coefficients obtenus sont ensuite quantifiés : les composantes les moins importantes, souvent associées aux hautes fréquences, sont réduites ou supprimées. Cette étape permet de diminuer fortement la quantité de données à stocker ou à transmettre.
Dans notre cas, ce format est particulièrement intéressant car il permet de réduire la taille des données tout en conservant une qualité visuelle suffisante pour l'observation et l'exploitation des images dans le système global.
Par ailleurs, un fonctionnement en mode jour et nuit a été pris en compte. En conditions de faible luminosité, un éclairage infrarouge permet de maintenir la capture d'images.

3.3.3 Tests et Validation

Afin d'évaluer les performances de la chaîne d'acquisition d'image sur plateforme ESP32, nous avons réalisé une série de tests.
Tout d'abord, nous avons vérifié le format des images générées. Les résultats montrent que le module peut correctement générer des images au format JPEG, et que ces images peuvent être utilisées par les autres modules du système.
De plus, nous avons étudié l'influence de la résolution sur la taille des images. Les résultats montrent qu'avec l'augmentation de la résolution, la taille des images augmente également, passant de quelques kilooctets en basse résolution (QQVGA) jusqu'à environ 30 kB en haute résolution (XGA). Comme illustré sur la figure ci-dessous, une augmentation de la résolution entraîne une augmentation de la taille des images ainsi qu'un impact sur le temps de capture.

4 Mesures de la taille des images

La taille des images JPEG (KB) a été mesurée pour différentes résolutions. Pour chaque résolution, cinq images consécutives ont été capturées dans des conditions identiques. Les variations observées de la taille des images proviennent principalement de la compression JPEG et du contenu visuel de la scène.

Table 2: Taille des images JPEG selon la résolution (ESP32-CAM, OV2640)

Résolution	Largeur×Hauteur	Img 1 (KB)	Img 2	Img 3	Img 4	Img 5	Moy. (KB)
QQVGA	160×120	1.4	2.9	2.9	2.9	3.0	2.62
QVGA	320×240	4.3	4.3	4.3	4.3	4.3	4.30
VGA	640×480	11.7	7.2	12.0	21.6	12.1	12.92
SVGA	800×600	32.4	35.0	24.4	23.4	15.5	26.14
XGA	1024×768	29.8	34.4	30.9	29.9	34.4	31.88

5 Mesures du temps de capture

Le temps de capture a été mesuré au niveau du pilote en chronométrant la fonction `esp_camera_fb_get()` (du début de l'appel jusqu'au moment où un tampon d'image valide est renvoyé). Ainsi, les valeurs rapportées représentent le **temps d'acquisition pur** dans la chaîne caméra de l'ESP32. Elles **n'incluent pas** la transmission WiFi, le traitement HTTP, ni le traitement côté navigateur. Pour chaque résolution, cinq captures consécutives ont été réalisées et la moyenne est reportée. De faibles variations peuvent apparaître en raison du contrôle d'exposition du capteur, du tamponnage interne et de l'ordonnancement du système.

Table 3: Temps de capture de `esp_camera_fb_get()` selon la résolution (ESP32-CAM, OV2640)

Résolution	Largeur×Hauteur	Essai 1 (μs)	Essai 2	Essai 3	Essai 4	Essai 5	Moy. (μs)
QQVGA	160×120	25	25	23	28	28	25.8
QVGA	320×240	30	26	27	36	34	30.6
VGA	640×480	216	144	231	236	262	217.8
SVGA	800×600	89	98	126	211	123	129.4
XGA	1024×768	219	249	254	219	204	229.0

FIGURE 5 – Influence de la résolution sur la taille des images JPEG et le temps de capture

Nous avons également analysé le temps de capture des images. Les résultats montrent que le temps de capture est globalement faible, mais qu'il varie en fonction de la résolution. Cependant, la latence globale provient principalement de la transmission WiFi ainsi que du traitement côté client.
Enfin, nous avons testé la qualité des images à différentes distances de prise de vue. Les résultats montrent que la qualité est bonne à courte distance, mais qu'elle diminue progressivement lorsque la distance augmente.
Globalement, ces résultats montrent que le module caméra fonctionne correctement et qu'il répond aux besoins du projet en termes de qualité d'image et de transmission des données.

3.4 WP 1.4 — Conception et fabrication du boitier (responsable : Joris Menard)

3.4.1 Rappel des objectifs

Détailler objectifs et liste des taches.

3.4.2 Solution envisagé

Travail théorique et recherches

3.4.3 Tests et Validation

Tests et validation de vos taches.

3.5 WP bis — Transmission (responsable : Joris Menard)

3.5.1 Rappel des objectifs

Détailler objectifs et liste des taches.

3.5.2 Solution envisagé

Travail théorique et recherches

3.5.3 Tests et Validation

Tests et validation de vos taches.

3.6 WP 2.1 — Stockage distant (responsable : Davy Clark)

3.6.1 Rappel des objectifs

Détailler objectifs et liste des taches.

3.6.2 Solution envisagé

Travail théorique et recherches

3.6.3 Tests et Validation

Tests et validation de vos taches.

3.7 WP 2.2 — Interface web (responsable : Xueying Xu)

3.7.1 Rappel des objectifs

Le work package WP2.2 correspond à l'interface web du système. Son objectif principal est de permettre à l'utilisateur de consulter et gérer les données collectées par le piège caméra de manière simple et accessible via un navigateur web. Plus concrètement, les tâches associées sont :

- Afficher les images capturées par le module caméra
- Permettre un accès au système via une interface web (adresse IP / site)
- Associer aux images certaines informations (date, heure, etc.)

— Visualiser l'état du système (caméra, batterie, etc.)
— Proposer différentes fonctionnalités (visualisation des images, gestion des caméras, journal système, etc.)
— Offrir une interface claire et intuitive pour l'utilisateur

Cette interface constitue le lien entre le système embarqué et l'utilisateur final.

3.7.2 Solution envisagé

Travail théorique et recherches

3.7.3 Tests et Validation

Tests et validation de vos taches.

3.8 WP 2.3 — Classification animalière par IA (responsable : Ewen Le Callet)

3.8.1 Rappel des objectifs

Détailler objectifs et liste des taches.

3.8.2 Solution envisagé

Travail théorique et recherches

3.8.3 Tests et Validation

Tests et validation de vos taches.

4 Bilan du projet

4.1 Bilan technique

4.2 Bilan personnel

5 Conclusion

Bibliographie

Références

[1] Project Management Institute, *A Guide to the Project Management Body of Knowledge (PMBOK Guide)*, 7e édition, 2021.

Annexes

A Répartition détaillée des Work-Packages

Cette annexe peut contenir un tableau de suivi des tâches par membre, avec les dates de début, d'échéance et le statut d'avancement.

B Exemples de données collectées

Cette annexe peut présenter des exemples de relevés (température, horodatage, détection de mouvement), ainsi que des captures d'écran de l'interface web.